Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

Kapitel 04 - VHDL - Entwurf digitaler Schaltungen

Prof. Dr.-Ing. Guillermo Payá Vayá, 3. Dezember 2024

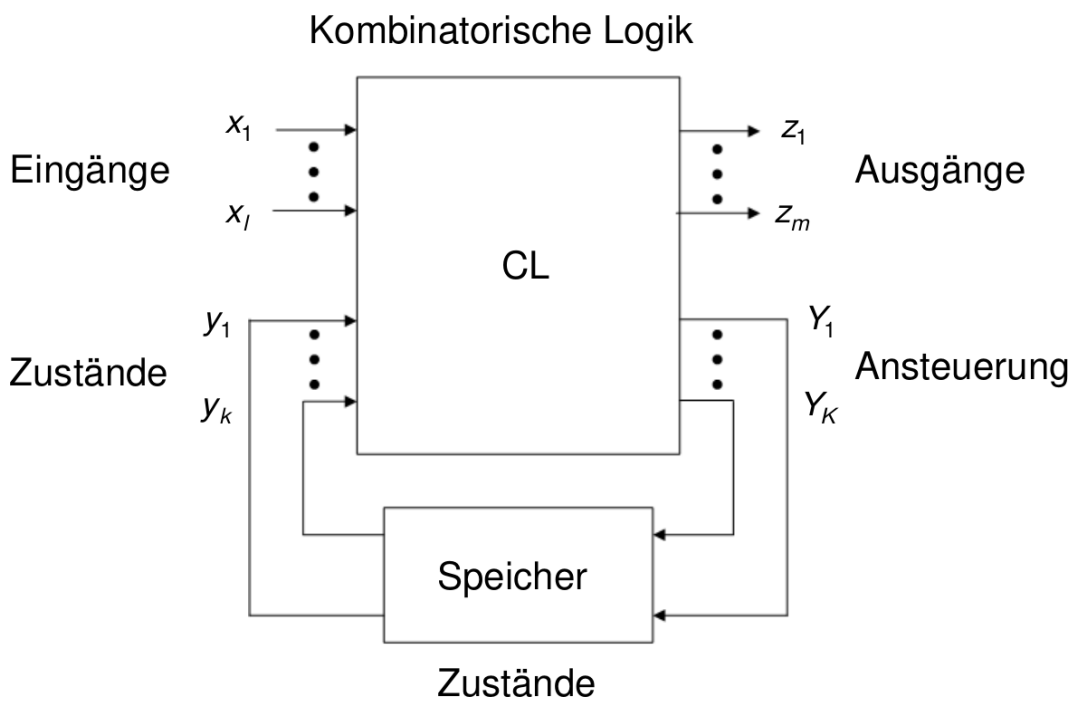
Entwurf von Zustandsautomaten

Endliche Automaten

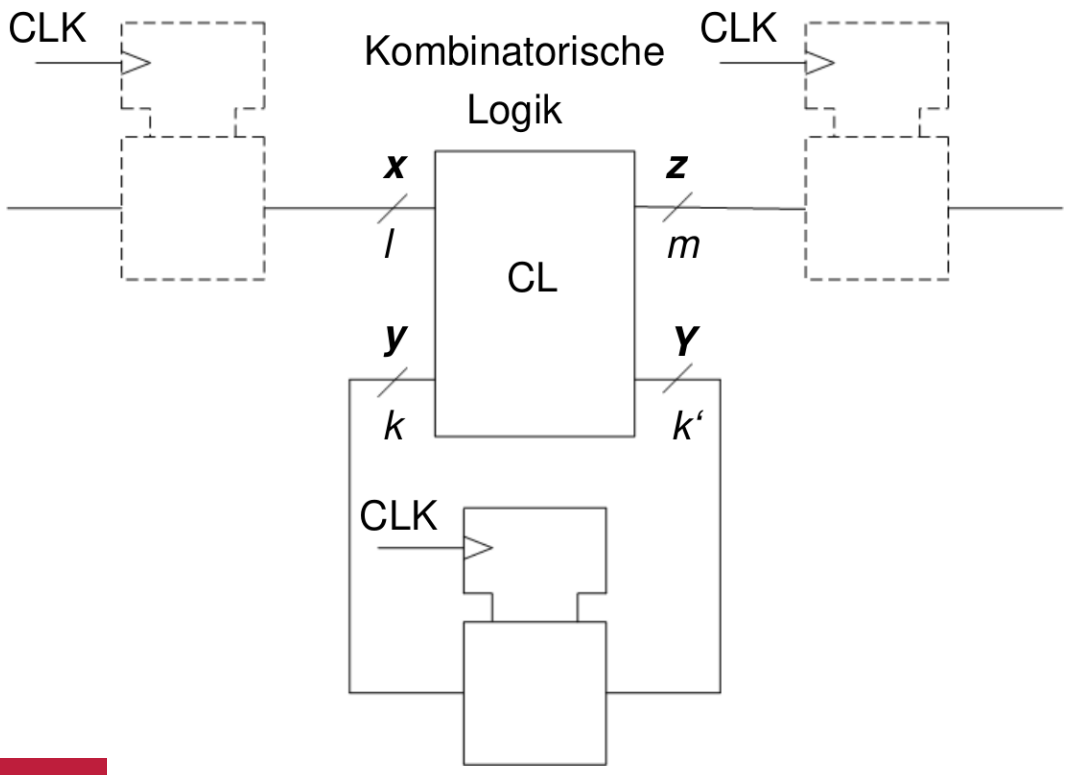
- Für die Implementierung von Steuerungen werden Anordnungen benötigt, die in Abhängigkeit von der Vergangenheit unterschiedlich auf aktuelle Eingangssignale reagieren.

  $\rightarrow$ Die Vergangenheit formt einen *Zustand* des Systems
- Im Falle einer endlichen Anzahl unterschiedlicher Zustände spricht man von **endlichen Automaten** oder **Zustandsautomaten** (English: ***finite state machine (FSM)***)
- *Synchrone endliche Automaten sind sequenziell arbeitende Logikschaltungen, die gesteuert durch ein periodisches Taktsignal eine Abfolge von Zuständen zyklisch durchlaufen.*
- Für den Entwurf und die Beschreibung von digitalen Systemen bilden Zustandsautomaten eine wesentliche Grundlage.

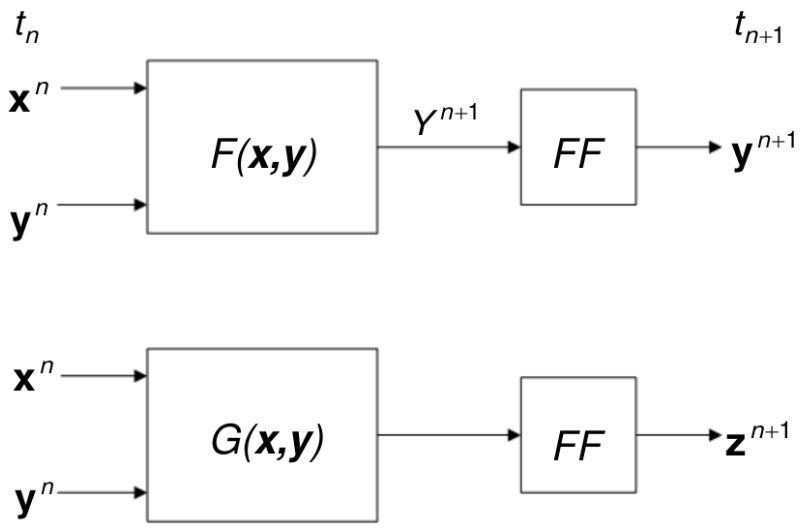
Prinzipielle Struktur eines endlichen Automaten



Synchron getakteter Automat



Logikfunktionen eines Automaten



$F(\cdot)$: Zustandsübergangsfunktion

$G(\cdot)$: Ausgabefunktion

Charakterisierung von Automaten

- Ein **Moore-Automat** ist eine FSM, in dem die Ausgangssignale **z** des Systems nur vom aktuellen Zustand **y** abhängen.

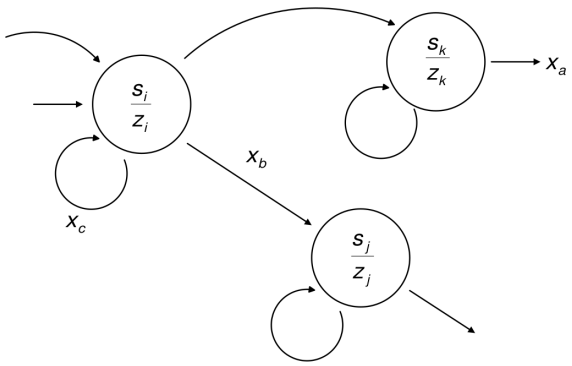
$$y^{n+1} = F(x^n, y^n)$$
$$z^{n+1} = G(y^n)$$

- Ein **Mealy-Automat** ist eine FSM, in dem die Ausgangssignale **z** des Systems vom aktuellen Zustand **y** und von den Eingangssignalen **x** abhängen.

$$y^{n+1} = F(x^n, y^n)$$
$$z^{n+1} = G(x^n, y^n)$$

Moore-Automat

- **Zustandsübergangsgraph**



- **Zustandsübergangstabelle**

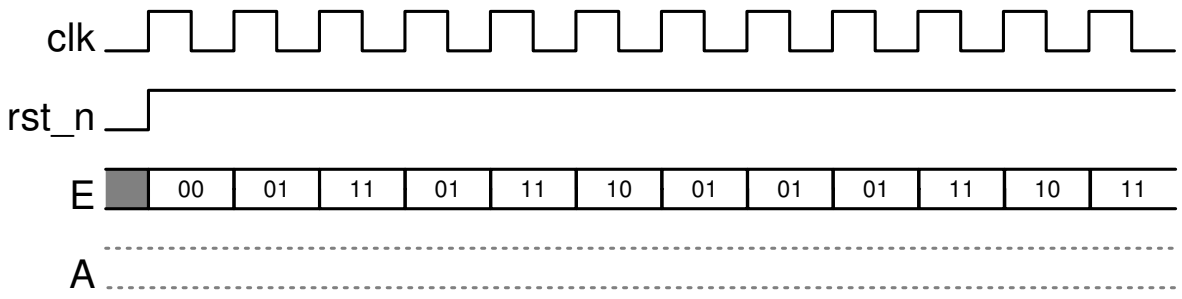
Eingabewert: x^n — Ausgabewert

s^n	x_a	x_b	---	z^n
s_0	s_i		---	z_i
s_1				
⋮	⋮	⋮		⋮
s_k		s_j	---	z_j

aktueller Zustand — nächster Zustand

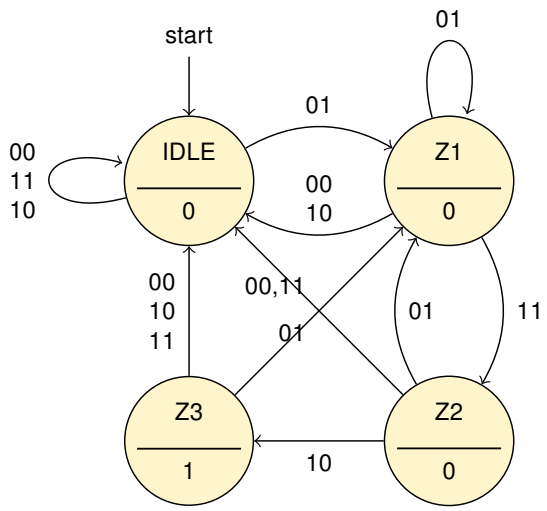
Impulsfolgenerkennung mit Moore-Automat (I)

- Beispiel: Ein 2-Bit-breiter Vektor `E` ist das Eingangssignal eines Systems. Ein Moore-Automat soll die Folge `01,11,10` am Eingangssignal erkennen und daraufhin mit dem Ausgangssignal `A` = `1` die Erkennung signalisieren. Der High-Pegel des Ausgangssignals `A` soll für eine Taktperiode verfügbar sein.



Impulsfolgenerkennung mit Moore-Automat (II)

- **Zustandsübergangsgraph**



- **Zustandsübergangstabelle**

s^n	00	01	10	11	z^n
IDLE	IDLE	Z1	IDLE	IDLE	0
Z1	IDLE	Z1	IDLE	Z2	0
Z2	IDLE	Z1	Z3	IDLE	0
Z3	IDLE	Z1	IDLE	IDLE	1

Impulsfolgenerkennung mit Moore-Automat (III)

```
entity FSM_MOORE is
  port(clk, rst, enable : in  std_ulogic;              -- Steuersignale
       E        : in  std_ulogic_vector(1 downto 0); -- Eingangsvektor
       A        : out std_uloigc);                    -- Ausgangssignal
end FSM_MOORE;

architecture RTL of FSM_MOORE is
  type zustaende_t is (IDLE, Z1, Z2, Z3);              -- Aufzählungstyp
  signal zustand_ff,zustand_nxt : zustaende_t;         -- Prozess-Kommunikation
begin

  FSM_SPEICHER: process(clk, rst_n)                    -- Zustandsaktualisierung
  begin
    if rst_n = '0' then
      zustand_ff <= IDLE;
    elsif clk = '1' and clk'event then
      zustand_ff <= zustand_nxt;
    end if;
  end process FSM_SPEICHER;

  ...
```

Moore-Automat

Impulsfolgenerkennung mit Moore-Automat (IV)

```
  ...

  FSM_ZUSTANDSUEBERGANG: process(E, zustand_ff)        -- Folgezustandsberechnung
  begin
    zustand_nxt <= IDLE; -- defaultzuweisung
    case zustand_ff is
      when IDLE => if E = "01" then  zustand_nxt <= Z1;
                   else              zustand_nxt <= IDLE;
                   end if;
      when Z1 => if E = "11" then    zustand_nxt <= Z2;
                 elsif E = "01" then zustand_nxt <= Z1;
                 else                zustand_nxt <= IDLE;
                 end if;
      when Z2 => if E = "10" then    zustand_nxt <= Z3;
                 elsif E = "01" then zustand_nxt <= Z1;
                 else                zustand_nxt <= IDLE;
                 end if;
      when Z3 => if E = "01" then    zustand_nxt <= Z1;
                 else                zustand_nxt <= IDLE;
                 end if;
    end case;
  end process FSM_ZUSTANDSUEBERGANG;

  ...
```

Moore-Automat

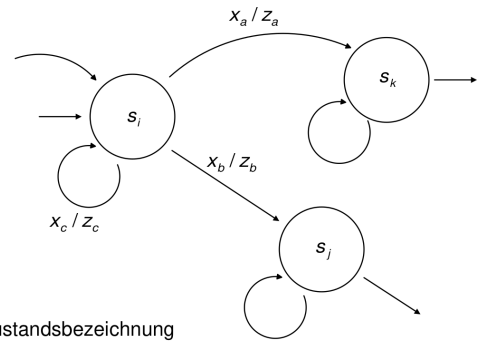
Impulsfolgenerkennung mit Moore-Automat (V)

```
  ...

  FSM_AUSGANG: process(zustand_ff) -- Ausgangsberechung
  begin
    case zustand_ff is
      when Z3 =>      A <= '1';
      when others => A <= '0';
    end case;
   end process FSM_AUSGANG;

end RTL;
```

Moore-Automat

Mealy-Automat

- **Zustandsübergangsgraph**



s_i Zustandsbezeichnung

x_a / z_a Eingabe / Ausgabe

- **Zustandsübergangstabelle**

Eingabewert

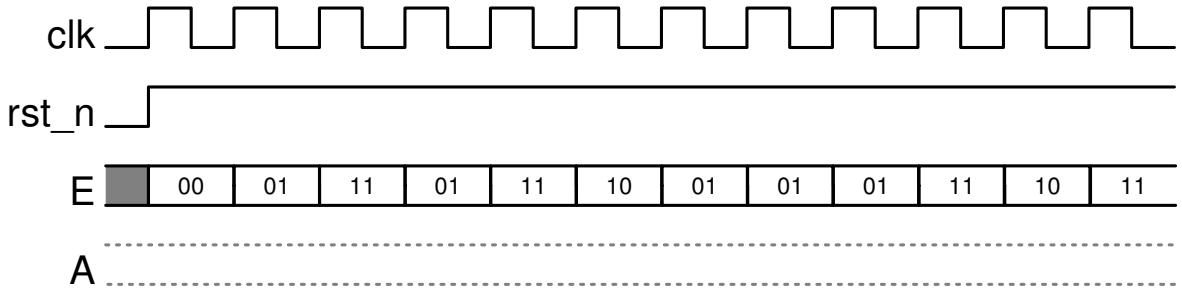
x^n

s^n	x_a	x_b	---
s_0	s_i, z_i		---
s_1			
$\vdots$	$\vdots$	$\vdots$	
s_k		s_j, z_j	---

aktueller Zustand

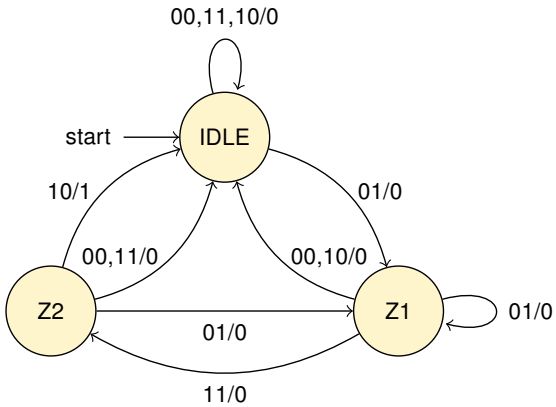
nächster Zustand, Ausgabewert

Impulsfolgenerkennung mit Mealy-Automat (I)



Impulsfolgenerkennung mit Mealy-Automat (II)

- **Zustandsübergangsgraph**



- **Zustandsübergangstabelle**

s^n	00	01	10	11
IDLE	IDLE/0	Z1/0	IDLE/0	IDLE/0
Z1	IDLE/0	Z1/0	IDLE/0	Z2/0
Z2	IDLE/0	Z1/0	IDLE/1	IDLE/0

Impulsfolgenerkennung mit Mealy-Automat (III)

```
entity FSM_MEALY is
  port(clk, rst, enable : in  std_ulogic;            -- Steuersignale
       E        : in  std_ulogic_vector(1 downto 0); -- Eingangsvektor
       A        : out std_ulogic);                   -- Ausgangssignal
end FSM_MEALY;

architecture RTL of FSM_MEALY is
  type zustaende_t is (IDLE, Z1, Z2);                  -- Aufzählungstyp
  signal zustand_ff,zustand_nxt : zustaende_t;        -- Prozess-Kommunikation
  signal a_nxt, a_ff            : std_ulogic;
begin

  FSM_SPEICHER: process(clk, rst_n)                    -- Zustandsaktualisierung
  begin
    if rst_n = '0' then
      zustand_ff <= IDLE;
      a_ff       <= '0';
    elsif clk = '1' and clk'event then
      zustand_ff <= zustand_nxt;
      a_ff       <= a_nxt;
    end if;
  end process FSM_SPEICHER;

  ...
```

Mealy-Automat

Impulsfolgenerkennung mit Mealy-Automat (IV)

```
  ...

  FSM_ZUSTANDSUEBERGANG: process(E, zustand_ff)       -- Folgezustandsberechnung
  begin
    zustand_nxt <= IDLE; -- defaultzuweisung
    case zustand_ff is
      when IDLE => if E = "01" then zustand_nxt <= Z1;
                   end if;
      when Z1 =>   if E = "11" then    zustand_nxt <= Z2;
                   elsif E = "01" then zustand_nxt <= Z1;
                   end if;
      when Z2 =>   if E = "01" then zustand_nxt <= Z1;
                   end if;
    end case;
  end process FSM_ZUSTANDSUEBERGANG;

  ...
```

Mealy-Automat

Impulsfolgenerkennung mit Mealy-Automat (V)

```
...

FSM_AUSGANG: process(E, zustand_ff) -- Ausgangsberechung
begin
  a_nxt <= '0';  -- Defaultzuweisung
  if (zustand_ff = Z2 and E = "10") then
    a_nxt <= '1'
  end if;
end process FSM_AUSGANG;

A <= a_ff;

end RTL;
```

Mealy-Automat

VHDL-Syntaxvarianten

Architekturvarianten, die aus einer unterschiedlichen Anzahl von Prozessen bestehen:

- **Ein-Prozess-Darstellung**
  - Zustandsübergänge und Ausgänge in einem getakteten Prozess
- **Zwei-Prozesse-Darstellung**
  - Ein getakteter Prozess zur Aktualisierung der Zustände
  - Ein kombinatorischer Prozess zur Bestimmung der Folgezustände und zur Bestimmung der Ausgänge
- **Drei- oder Mehr-Prozesse-Darstellung**
  - Ein getakteter Prozess zur Aktualisierung der Zustände
  - Ein kombinatorischer Prozess zur Bestimmung der Folgezustände
  - Ein oder mehrere, kombinatorische Prozesse zur Bestimmung der Ausgänge

Zustandscodierung

- Mit der Zustandscodierung wird den Namen der symbolischen Zustände eines Automaten, z.B. den Elementen eines Aufzählungstyps, eine Bitrepräsentation zugeordnet, sodass die Zustände durch eine Gruppe von D-Flipflops realisiert werden.
- Die Funktion des Automaten, d.h. dessen funktionales und zeitliches Klemmerverhalten, soll durch die Art der Zustandscodierung nicht beeinflusst werden.
- Die Ziele der Zustandscodierung sind:
  1. Eine hohe Takfrequenz,
  2. minimaler Ressourcenbedarf und
  3. ein störungssicherer Betrieb

Unsetzung der Zustandscodierung im VHDL-Code

- Neben der Möglichkeit, die Zustandscodierung dem Synthesewerkzeug zu überlassen (Kapitel 6), besteht auch die Möglichkeit, diese im VHDL-Code festzulegen.
- Durch Verwendung des VHDL-Attributs `enum_encoding` kann z.B. eine One-Hot-Zustandscodierung wie folgt vorgenommen werden:

```
type zustaende_t is (Z0, Z1, Z2, Z3);
attribute enum_encoding : string;
attribute enum_encoding of zustaende_t : type is "0001 0010 0100 1000";
signal zustaende_ff, zustaende_nxt : zustaende_t;
```

Codierung 1

- Alternativ kann eine explizite Codierung durch Zuweisung konstanter Bitvektoren vorgenommen werden:

```
subtype zustaende_t is std_ulogic_vector(3 downto 0);
constant Z0 : zustaende_t := "0001";
constant Z1 : zustaende_t := "0010";
constant Z2 : zustaende_t := "0100";
constant Z3 : zustaende_t := "1000";
signal zustaende_ff, zustaende_nxt : zustaende_t;
```

Codierung 2

Moore Output Buffering

- In einem typischen Moore-Automat benötigen wir eine kombinatorische Ausgangslogik, um die Ausgangsfunktion zu implementieren.
  - Eine Möglichkeit, die Auswirkungen der Ausgangslogik zu verringern, besteht darin, sie durch geschickte Zuweisung von Zuständen vollständig zu eliminieren.
  - Eine zweite Möglichkeit besteht darin, zusätzliche Registerbits im Zustandsspeicher entsprechend den Ausgangswerten zu belegen

Literatur

- Jürgen Reichardt, Bernd Schwarz, "VHDL-Simulation und -Synthese", Walter de Gruyter GmbH, 2020
- Peter J. Ashenden, "The Designer's Guide to VHDL", Morgan Kaufmann, 2010.
- Janick Bergeron, "Writing Testbenches: Functional Verification of HDL Models", Springer Science & Business Media, 2012.